

Chapter 11. Financial Data Workflows

Throughout this part of the book, you have explored the fundamental layers of the financial data engineering lifecycle: ingestion, storage, transformation and delivery, and monitoring. Adhering to this structured framework is crucial for building and maintaining efficient financial data infrastructures.

Importantly, as your company expands, the financial data infrastructure will become larger and more complex, generating many interdependencies and links between the different layers. To address this challenge, further refinement is required to build multiple independent and specialized data processing components. Data engineers call these components data pipelines or workflows.

This chapter will explore the foundational concepts of data workflows, introducing the ideas of workflow-oriented software architectures and workflow management systems. Then, it will cover the main types and characteristics of financial data workflows that can be implemented to handle financial data processing tasks.

Workflow-Oriented Software Architectures

Before explaining workflows, it's important to understand the business rationale behind their practicality. First, as a natural part of the growth journey of any digital company, the technological stack tends to increase in size and complexity, creating numerous interdependencies, logical flows, and interactions among various system components. For this reason, the software development community has identified the need for architectural designs and tools that organize software transactions into

structured workflows that can be defined, coordinated, managed, monitored, and scaled following a specific business logic. In this book, I will use the term *workflow-oriented software architecture* (WOSA) to describe this design pattern.

The concept of WOSA is highly applicable to financial markets, where many financial operations are organized as a series of actions carried out in a specific sequence. A notable example is financial transactions, which generally follow a defined workflow consisting of multiple steps.

Examples follow:

- The lifecycle of financial trades involves several steps such as trade initiation, order placement, trade execution, trade capture, enrichment, validation, verification, confirmation, clearing, and settlement.
- The lifecycle of credit card payments involves a chain of steps that may involve the customer, merchant, payment gateway, payment processor, acquiring bank, issuing bank, and card network. Along the way, the same payment goes through several steps that start with submission, authorization, balance and fraud checks, confirmation or rejection, clearing, and settlement.

In essence, WOSAs focus on organizing and executing complex processes through structured workflows, enhancing modularity, efficiency, and manageability in software systems. An important category within workflows is the data workflow, designed specifically for the processing and management of data. Let's examine some of its technical aspects and concepts in the following section.

What Is a Data Workflow?

A data workflow is a repeatable process that involves applying a structured sequence of data processing steps to an initial dataset, producing a desired output dataset.

Data workflows are fundamental to data engineering. As a financial data engineer, you will frequently build financial data workflows. To excel at this task, an important skill to learn is workflow abstraction. This in-

volves defining a conceptual framework for your workflow that disregards technological implementation details to guarantee generalization. It is similar to how concepts of data storage models and data storage systems were used in [Chapter 8](#) to abstract the specifics of storage technologies.

One of the most widely adopted data workflow abstractions is the [dataflow paradigm](#). Its core principle involves organizing an application into a directed computational graph. In this graph, nodes represent individual computation steps, while links express data dependencies or the flow of data between these computations. Typically, the computational graph underlying a workflow is designed without cycles. When structured in this manner, the graph is referred to as a Directed Acyclic Graph (DAG).

The most basic type of computational DAG is the linear DAG. It organizes tasks in a sequential order where each task N can have at most one preceding task ($N-1$) and one subsequent task ($N+1$). [Figure 11-1](#) illustrates a three-task linear computational DAG.

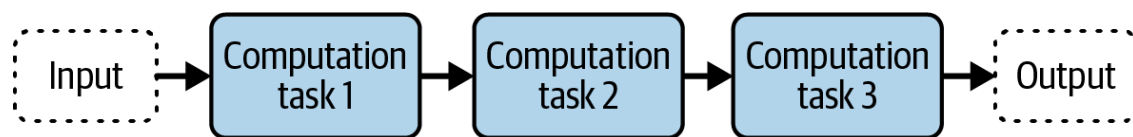


Figure 11-1. Linear DAG

Frequently, computational DAGs are more complex than the linear case. For example, a specific computing task may depend on multiple preceding tasks. [Figure 11-2](#) illustrates such a scenario, where computation task 6 relies on tasks 3, 4, and 5.

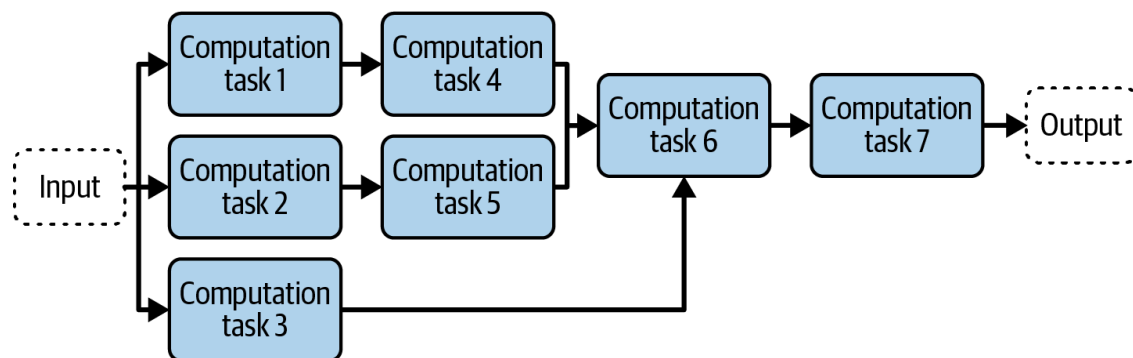


Figure 11-2. Complex DAG

When defining a computational DAG, a fundamental question arises: “What constitutes a task?” While there isn’t a universal answer to this question, several best practices have been suggested. For example, a task needs to be an atomic unit of execution, meaning that it either succeeds or fails as a whole. Furthermore, it is considered a best practice to ensure that tasks are idempotent, meaning that running the same task multiple times will yield the same result as running it once, without causing any unexpected or undesirable side effects. This can be useful since it can allow you to add task checkpoint features to your DAG, allowing you to resume DAG execution from the failed step rather than execute it all over again.

Another consideration is the size of tasks within a computational DAG. They should strike a balance: they need to be small enough to facilitate debugging and traceability, but not so small that they introduce unnecessary overhead and degrade DAG performance. Conversely, overly large tasks simplify the DAG structure but can constrain debuggability and lead to costly retry operations.

Moreover, a good practice is to associate tasks with business logic. This can involve basic tasks such as data quality checks or more complex computations like modeling and analytics.

Workflow Management Systems

Once a workflow abstraction has been defined, the next step is to design a system that provides the infrastructure and tools necessary for building, orchestrating, and monitoring a WOSA. In this context, I’ll use the term *workflow management system* (WMS) to refer to such systems. When developing a WMS, various properties must be considered. In the following subsections, I will briefly discuss the most common ones.

Flexibility

A flexible WMS is capable of managing a diverse range of tasks and workflows. For instance, it should support the creation of simple linear work-

flows as well as more complex ones. Additionally, it should offer flexibility in how workflows are initiated, allowing for scheduled executions as well as triggering based on specific events, such as the arrival of files or data in designated storage locations.

Configurability

A configurable WMS enables users to define workflow specifications according to their needs. For instance, many WMSs allow users to define workflows and tasks using infrastructure-as-code (IaC) methodologies. This can be achieved through programming languages like Python or Domain-Specific Languages (DSLs) such as JSON.

With IaC, users can programmatically define workflow and task specifications such as dependency structure, input/output parameters, error handling, timeout policy, retry logic, status update, workflow triggers, concurrent execution limit, logging, and many more. [Figure 11-3](#) illustrates a typical approach to workflow and task definition, which contains all the information necessary to define the behavior of a workflow.

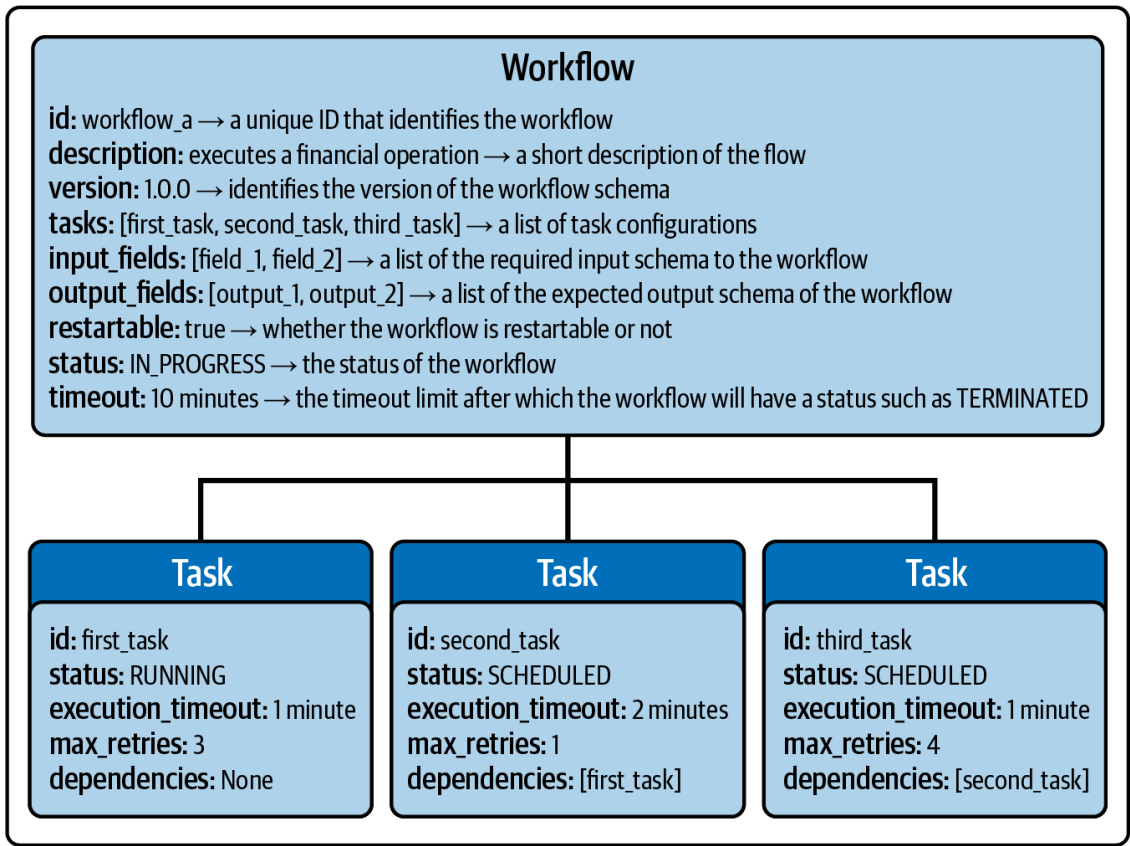


Figure 11-3. A workflow and task definition template

Dependency Management

WMSs vary in how they manage and configure dependencies between tasks and workflows. In simple WMSs, users are only able to define static and linear workflows. However, more advanced WMSs may allow for parallel workflow/task execution, dynamic and conditional task creation, and information passing between tasks. To illustrate, (a) in [Figure 11-4](#) displays a workflow where tasks 3 and 4 are conditionally executed after task 2, depending on whether a specific condition is satisfied; if not, task 5 follows task 2. In contrast, (b) in [Figure 11-4](#) illustrates a workflow that dynamically generates a variable number of tasks (N) at runtime, based on the input received from task 1.

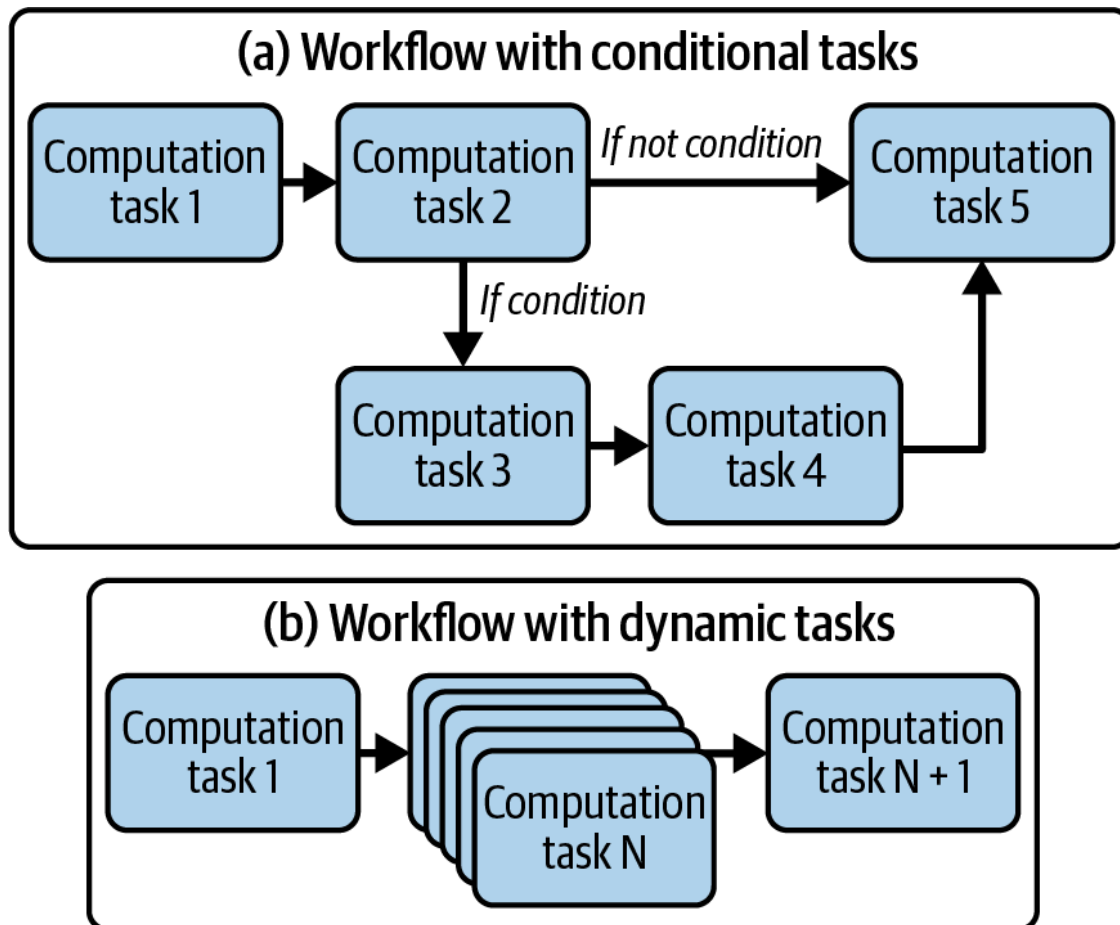


Figure 11-4. Workflow with conditional tasks (a), and workflow with dynamic tasks (b)

Coordination Patterns

In every WMS, components like workflows and tasks are designed to interact with each other. Various communication patterns have been developed to describe how these interactions occur. Two notable patterns are

the *Synchronous Blocking Pattern* (SBP) and the *Asynchronous Non-Blocking Pattern* (ANBP).

The SBP involves one component calling another and waiting for a response, which is typical in systems using HTTP and RESTful APIs. In a WOSA relying on an SBP, each task within a workflow is executed sequentially within the same process, with each task waiting for the previous one to complete before proceeding.

Conversely, the Asynchronous Non-Blocking Pattern operates in a fire-and-forget mode, where a component sends a request or message to another component without waiting for a response or confirming its processing. In a WMS, this pattern can be implemented by running each workflow task independently on separate processes or machines and having a dedicated task queue that gathers incoming messages for execution. [Figure 11-5](#) provides a visual representation of this concept.

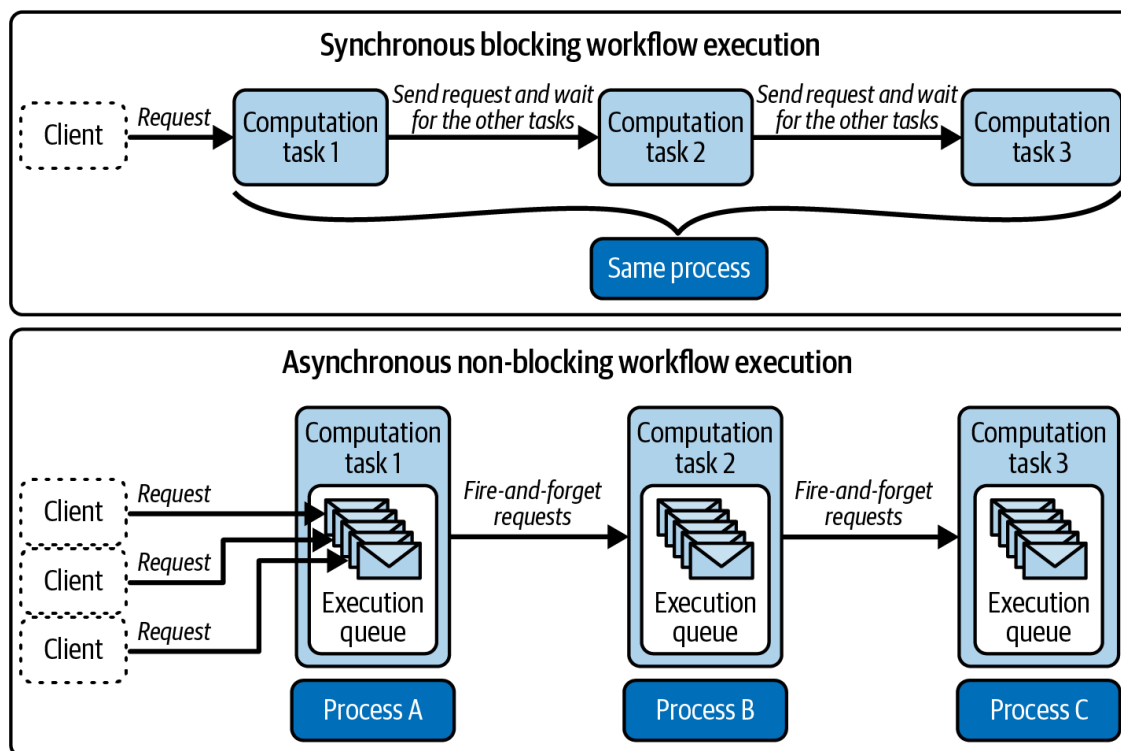


Figure 11-5. Synchronous Blocking Pattern versus Asynchronous Non-Blocking Pattern

Scalability

WMS scalability is typically assessed based on its capacity to handle concurrent executions of workflows and tasks. The scalability requirements

vary depending on the nature of the WMS. Event-driven and high-volume WMSs, for instance, demand robust scalability capabilities to manage large volumes of tasks triggered by real-time events or continuous streams of data. In contrast, scheduled batch-oriented WMSs may have less stringent scalability needs as they operate on predefined schedules and process tasks in batches.

Integration

A highly desired feature of WMSs is their capability to seamlessly integrate with other tools and technologies, especially those that the WMS is designed to manage or coordinate. For example, when orchestrating applications in the cloud, it might be more convenient to rely on a cloud-based WMS as it will easily integrate with the cloud services running the applications.

Many WMSs incorporate operators, which are predefined tasks enabling seamless integration with a variety of technologies. These operators facilitate common and essential operations within workflows. For instance, a database operator allows users to establish connections and interact with specific types of databases like Postgres, Redshift, Snowflake, and BigQuery. This capability streamlines workflow execution by providing ready-made functionalities for interacting with diverse technological environments.

Now that you have established a foundational understanding of workflows, let's explore some of the most common types of data workflows, outlining their key characteristics, the technologies they employ, and their applications in the financial domain.

Types of Financial Data Workflows

This section explores four fundamental types of data workflows that are central to financial data engineering: *Extract-Transform-Load (ETL) workflows* streamline data movement and transformation; *microservices workflows* facilitate modular and scalable application architectures; *machine*

learning workflows automate model training and deployment pipelines; and *streaming workflows* enable real-time data processing and analytics. Each type plays a critical role in enhancing efficiency and responsiveness across diverse data-driven applications and industries. Let's explore each of these workflow types in more detail.

Extract-Transform-Load Workflows

The predominant type of data workflow is Extract-Transform-Load (ETL), a three-phase workflow in which data is retrieved from one or more sources, transformed using predefined business logic, and saved in a target destination.

In a typical ETL workflow, raw data files are initially extracted and stored in a staging area within a data lake. Following this, various transformations are applied to each file. This can range from data quality checks and cleaning operations (e.g., drop duplicates, handle null values, remove erroneous records, standardize fields, etc.) to computation and data enrichment steps (e.g., scaling, normalization, feature engineering, etc.). Finally, transformed data is stored in the enterprise data warehouse, which serves as the central repository from which data consumers access and utilize the data they need.¹ [Figure 11-6](#) illustrates this ETL use case.

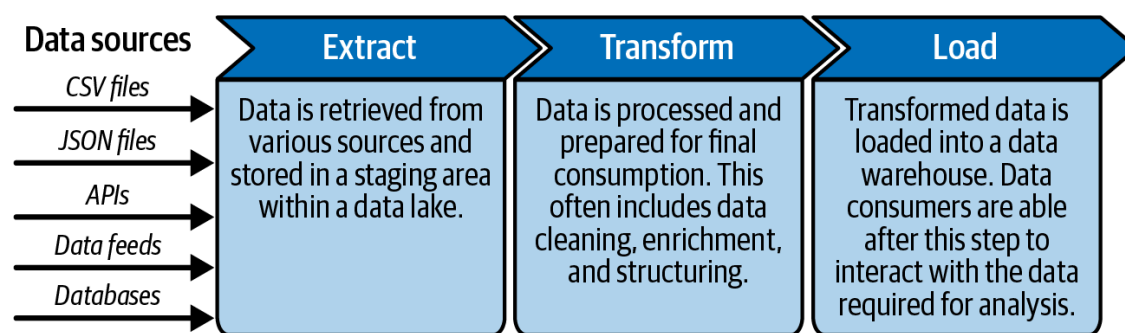


Figure 11-6. Illustration of the ETL workflow

ETL workflows may vary in terms of complexity and requirements. Traditional ETL workflows are often built as linear DAGs that process data in batches with a predefined schedule (e.g., daily or weekly). However, challenges can arise when new business problems create additional scalability and performance requirements. For example, you might need to implement parallel processing in your workflow to accommodate

larger data volumes. This can involve, for example, splitting a large file into small chunks and transforming each into a separate dynamic task. Another major problem occurs when new types of data arrival processes must be handled. Nowadays, it is common to have data arriving in regular batches, as well as single data entries generated through event-based mechanisms.²

A large variety of ETL WMSs are available. These include commercial enterprise tools (e.g., IBM DataStage, Oracle Data Integrator, Talend, Informatica), open source tools (e.g., Apache Airflow, Prefect, Mage, Apache Spark), cloud-based tools (e.g., AWS Glue, Google Dataprep), and custom tools. These tools vary in terms of the features that we illustrated previously in [“Workflow Management Systems”](#).

With so many ETL tools available, it can be overwhelming to choose the right one. I recommend starting by defining your workflows and identifying the features you need based on your business requirements. Subsequently, evaluate the various options in the context of your existing technological stack. For example, if you are running all of your stack on AWS, then you might benefit from using Amazon Managed Workflows for Apache Airflow or AWS Glue. Importantly, I highly recommend not using a complex tool to perform basic ETL operations. It might be that all you need is a simple job scheduler that runs a Python script on a provisioned server or managed services such as AWS Lambda or AWS Batch, or Python shell jobs in AWS Glue.

Apache Airflow, developed and released by Airbnb in 2015, has become the leading open source tool for data workflow management. Airflow can be deployed as a containerized application and is also available as a managed service from cloud providers like Google and Amazon.

Airflow's high popularity is due to its extensive feature set and the frequent addition of new features and optimizations. It functions as an IaC tool, allowing workflows to be defined as DAGs written in Python. In addition, it offers a wide range of operators, triggers, customizable plug-ins, a flexible flow design (dynamic tasks, cross-DAG dependencies), SLA features, a security model, a rich UI, and operational control options.

Apache Airflow is fundamentally an orchestration engine with great versatility. It's designed to schedule, manage, and monitor different types of workflows, including data pipelines, machine learning pipelines, batch processing pipelines, reporting and monitoring pipelines, data migration, and data quality checks.

Like most tools, Airflow has several drawbacks to consider. First, while Airflow supports passing data between tasks using XCOM, it still does not offer a highly reliable and scalable method for cross-task data sharing. Second, Airflow is primarily designed for batch processing and does not natively support real-time or event-driven data streams. Third, being a workflow orchestrator rather than a dedicated data processing framework, Airflow lacks some performance and scalability features required for handling large volumes of data. In such cases, integrating specialized tools like Apache Spark for intensive data processing within an Airflow DAG can be a viable option.

ETL workflows are essential for financial institutions. They are extensively used to automate the extraction of subscription-based data from financial data vendors. Another common use case is data aggregation, where various types of data are periodically retrieved from multiple sources and consolidated into a single repository, such as a data warehouse. ETL workflows also automate the generation of risk and analytical

reports for internal use, dashboarding systems, and compliance. Last but not least, they are used in financial research involving historical data analysis and processing, financial analysis, risk calculations, portfolio reconciliation, and performance tracking.

Stream Processing Workflows

A stream processing workflow consists of a sequence of data processing steps where data flows as a continuous stream of events and is processed in real time, enabling immediate reactions to new information. [Figure 11-7](#) illustrates the concept visually.

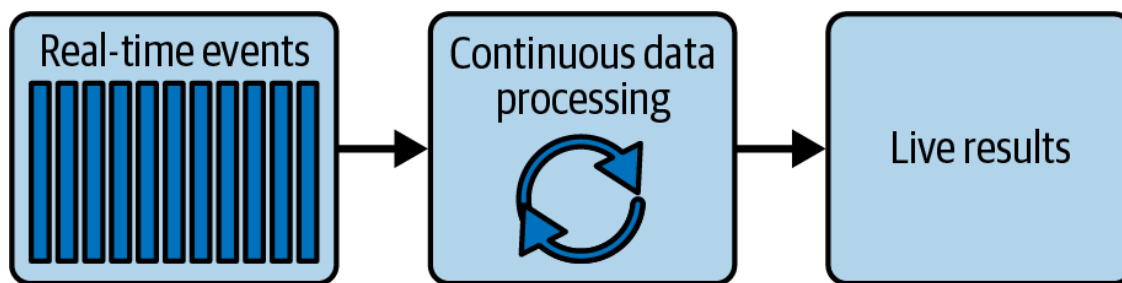


Figure 11-7. Illustration of a basic stream processing workflow

Stream workflow management solutions prioritize scalability for large data streams, fast transformations, in-memory computing, event-driven capabilities, and asynchronous non-blocking data processing.

A typical streaming architecture consists of four main components:

Ingestion layer

Handles the real-time reception of incoming data traffic from various sources. For example, a dedicated application can be deployed on AWS EC2 and load-balanced using AWS Elastic Load Balancing (ELB).

Message broker

Stores the event stream and acts as a high-performance intermediary between data producers and consumers. Examples include Apache Kafka and Google Pub/Sub.

Stream processing engine

Consumes data from a topic and applies transformations, analytics, or checks. Examples include Apache Storm, Apache Flink, Spark Streaming, and cloud functions such as AWS Lambda.

Data storage system

Persists the data for further consumption. Examples include Apache Cassandra, AWS DynamoDB, Google Bigtable, and Google Firestore.

[Figure 11-8](#) illustrates this architecture.

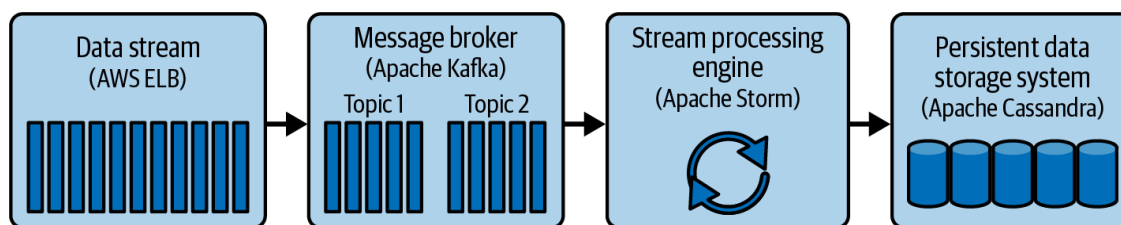


Figure 11-8. Example of a typical stream processing architecture

More complex stream workflow architectures can be built. One common pattern is the *lambda architecture*, which typically consists of three layers:

Speed layer

Handles real-time stream processing and creates real-time views for instant insights.

Batch layer

Consumes the same data stream to perform historical analysis, persisting a reliable view of the data (batch view).

Serving layer

Responds to queries made to both the speed and batch views.

The three layers can be implemented in various ways, but a basic illustration is shown in [Figure 11-9](#). As depicted, data is first received by AWS ELB and then ingested into a Kafka topic. Once the new data enters Kafka,

it is simultaneously consumed by the batch and speed layers' consumer groups. The speed layer generates real-time views for low-latency access to recent data, while the batch layer operates in a lower latency but higher throughput mode. This ensures that data is prepared, cleaned, pre-processed, aggregated, checked against existing historical data, and reliably persisted for later use.

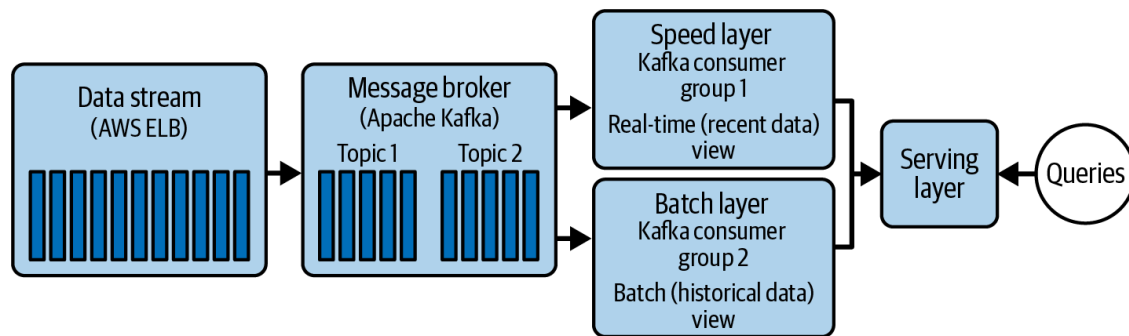


Figure 11-9. Illustration of a lambda architecture

Lambda architectures are often criticized for their complexity and the maintenance overhead they entail. A simpler alternative, known as the *kappa architecture*, integrates batch and real-time processing into a unified workflow that treats all data as a continuous stream. This approach leverages a single high-performance stream processing engine, simplifying both implementation and management.³ [Figure 11-10](#) provides an example. Initially, data is received by AWS ELB and then ingested into a Kafka topic. Once the data enters Kafka, it is consumed by the stream processing layer, which utilizes an Apache Flink engine to process these events in real time, performing various transformations. The processed data is subsequently stored in a Cassandra database, allowing for efficient querying and further analysis.

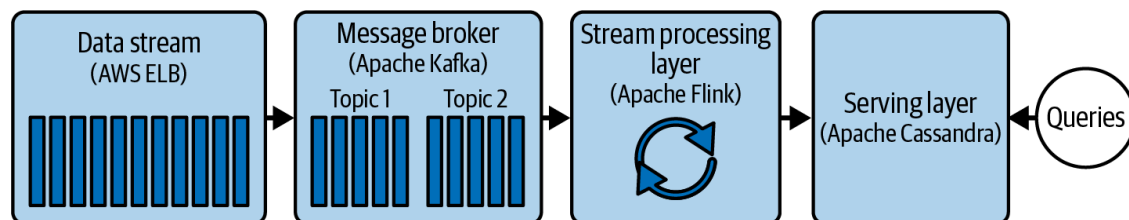


Figure 11-10. A kappa architecture

Stream processing finds numerous applications within the financial sector, with one prominent use case being real-time fraud detection and ana-

lytics. Financial institutions face significant threats from payment and credit card fraud, as highlighted by the [2023 Association for Financial Professionals Payments Fraud and Control Survey report](#), underwritten by JPMorgan, which revealed that 65% of participants experienced payment fraud attacks or attempts in 2022. Effective fraud detection systems necessitate automation to reduce manual workload, high accuracy to minimize false positives and maintain customer satisfaction, scalability to handle large transaction volumes, and speed for real-time assessment and transaction execution.

Real-time stream processing architectures provide a robust solution to tackle these challenges. For instance, combining Apache Kafka as a messaging stream broker with Apache Spark ML for predictive modeling and Apache Spark Streaming for real-time stream processing can be highly effective. Similar architectures leveraging tools like Apache Flink and Apache Storm also offer robust alternatives.⁴ Managed cloud streaming solutions cater specifically to these needs, such as Azure Stream Analytics, which includes [built-in machine learning capabilities for anomaly detection](#), Amazon Managed Services for Apache Flink, and Google Cloud Dataflow. In addition, AWS provides Elastic MapReduce (EMR), a versatile platform enabling users to build and manage big data environments encompassing Apache Spark and Apache Flink, among other tools.

Another application of stream processing in financial markets is for powering data feeds. Streaming architectures can be used to efficiently ingest, process, and analyze real-time market data sourced from stock exchanges, data providers, and trading platforms. Furthermore, financial institutions are progressively embracing streaming solutions to update their data infrastructure. This involves migrating diverse operational data from legacy mainframe systems to cloud-based platforms, a process known as *mainframe offload*.

Microservice Workflows

In [Chapter 6](#), you were introduced to the concept of microservices. As a reminder, microservices are small, self-contained, and independently de-

ployable applications that work together. There is a growing trend toward using microservice architectures to effectively implement and bring business ideas to life. Crucially, for microservices to collaborate effectively, coordination is essential. This typically involves defining and managing microservice workflows, which I will be discussing in this section.

Prior to designing a microservice workflow, it's crucial to assess several factors. First of all, determining what qualifies as a microservice requires careful consideration from both technical and business perspectives. From a technical standpoint, it's important to strike a balance between high cohesion and low coupling. This means minimizing dependencies between microservices while ensuring that related application logic remains cohesive within each microservice. [Figure 11-11](#) illustrates the concept.

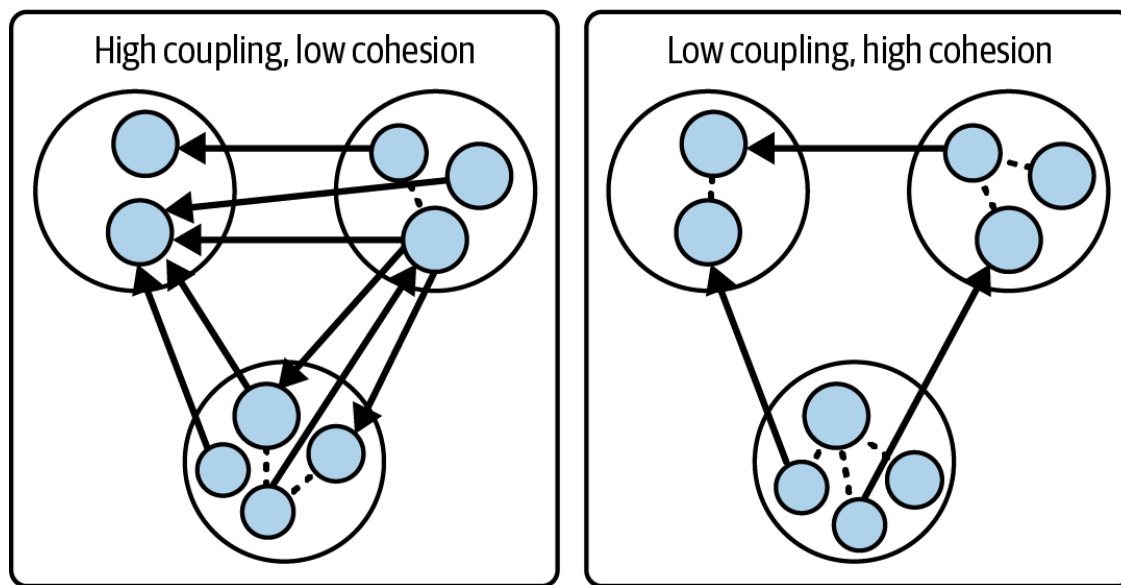


Figure 11-11. Coupling versus cohesion

From a business standpoint, a significant challenge involves deciding how to formalize and integrate business logic across multiple microservices. Various approaches exist for this purpose, including object-oriented principles such as the Single Responsibility Principle (SRP), procedural patterns like the transaction script pattern, and comprehensive frameworks such as Domain-Driven Design (DDD).

DDD is quite elegant and provides a reliable approach for designing business logic in a microservice architecture. In DDD, developers design the

so-called *domain models* to capture and represent the business requirements and unique components of a domain, establishing a conceptual basis for building software applications. A business model includes the entities, relationships, rules, and logic that define a specific domain or problem space within an application.

DDD strives to establish a *ubiquitous language* that fosters shared understanding among domain experts and developers. Ubiquitous language helps bridge the communication gap between technical and nontechnical stakeholders, facilitating more effective collaboration and a clearer definition of requirements.

Another essential concept in DDD is *bounded context* (BC), which can be thought of as the setting that defines the meaning of a word. Just as “bond” can refer to different things depending on the context, a BC establishes clear and distinct definitions for terms and concepts within a specific domain area. This helps break down complex domains into manageable subdomains, ensuring clarity and alignment with business needs and allowing developers to focus on a specific area of the domain without being overwhelmed by the complexity of the entire domain.⁵

After you have defined and created your microservices, the next step is to organize them in a workflow. A microservice workflow coordinates multiple microservices to ensure they operate following a specific business logic, thereby achieving cohesive application functionality. To illustrate the idea, let’s consider a simple example of an online store order processing system based on microservices, as depicted in [Figure 11-12](#). As shown in the figure, when a customer submits a purchase order, six microservices collaborate to manage the various stages of the order lifecycle.

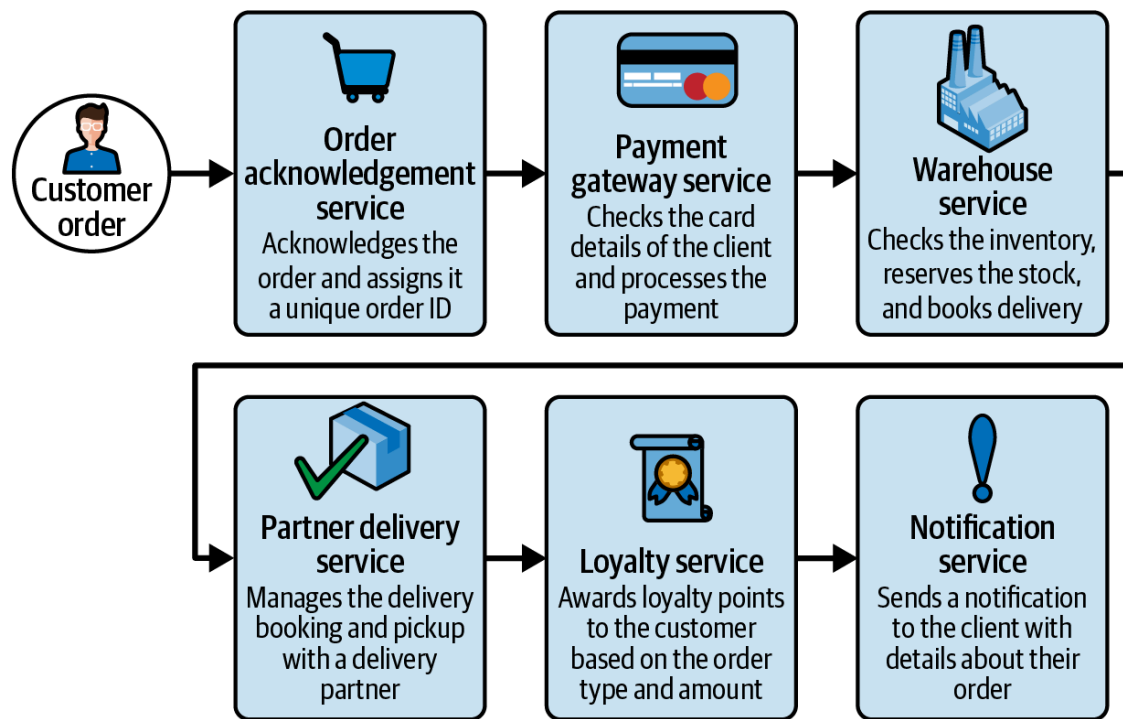


Figure 11-12. Microservice-oriented online store order processing system

In real-world scenarios, microservice workflows frequently exhibit greater complexity than the linear structure depicted in [Figure 11-12](#). This happens when microservices are deployed in a distributed system. Here, establishing reliable communication mechanisms becomes critical for coordinating microservices and maintaining workflow consistency. Traditional application designs typically rely on ACID database transactions for this purpose. However, in distributed microservice systems, it is technically cumbersome to implement distributed ACID transactions.

As a reliable alternative, microservice engineers often rely on a popular design pattern known as the [saga pattern](#). A saga organizes a microservice workflow into a sequence of local transactions, where each transaction executes its logic and communicates with the next via update messages or events. Two primary approaches exist for coordinating a saga: choreography and orchestration.

In choreography-based sagas, microservices directly communicate and coordinate with each other to complete the workflow. Each microservice acts as both a participant and coordinator within the collaborative process. This means that each microservice needs to be aware of the business process, the services with whom to interact, the tasks to execute, and the messages to exchange.

On the other hand, orchestration-based sagas introduce a central orchestrator to manage the workflow execution. In this approach, participating microservices do not need to be aware that they are part of a larger orchestration process. Instead, they simply follow instructions provided by the orchestrator. Orchestration-based sagas commonly leverage a message broker to facilitate communication and ensure the orderly execution of tasks across distributed microservices.

Let's look at the two saga patterns. Part (a) in [Figure 11-13](#) represents a choreography-based saga. In this instance, services interact directly with each other using specific topics. For example, Services A and B communicate with Service E via Topic E, while Services E and F use Topic B to interact with Service B, and Service C communicates with Service G through Topic G. In contrast, (b) in [Figure 11-13](#) illustrates an orchestration-based saga, where all service communication is mediated by a central orchestrator. Each service receives instructions from the orchestrator via its designated topic and communicates back to the orchestrator through the same topic.

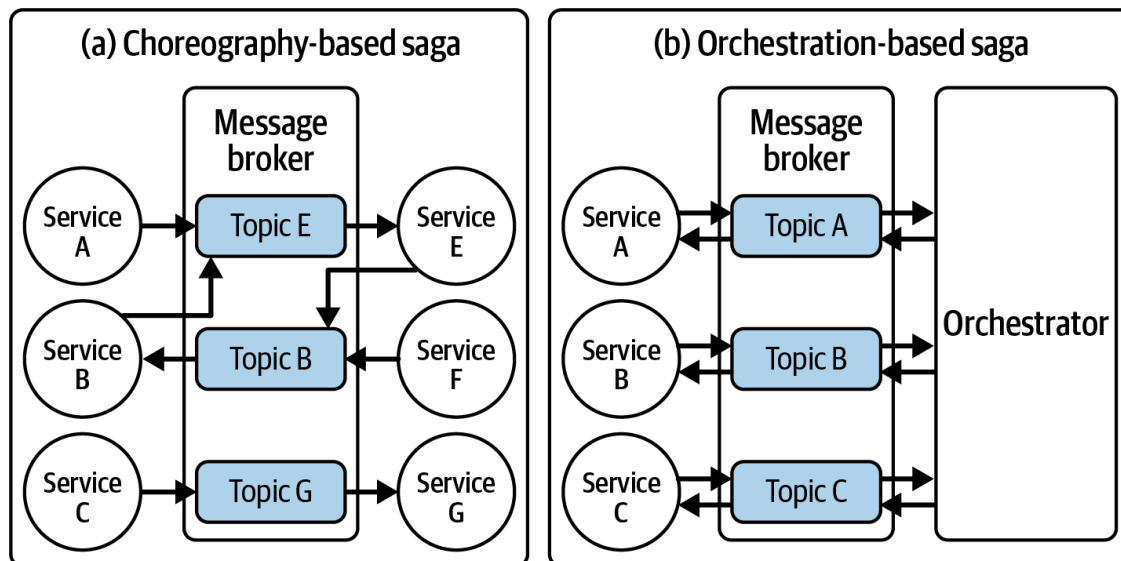


Figure 11-13. Choreography-based saga (a), and orchestration-based saga (b)

In terms of technological implementation, a microservice workflow management system can be designed in various ways. If we consider the more prevalent orchestration-based approach, a fundamental setup typically includes three components:

- A core orchestration engine responsible for managing workflows and facilitating communication among microservices. This engine could be a custom-built application using languages like Java or Python, an open source tool such as Orkes Conductor, or a cloud-managed solution such as AWS Step Functions or Google Workflows.
- A backend database to store details about workflow executions, ideally suited for OLTP systems such as PostgreSQL.
- A message broker for handling the queuing and exchange of messages between microservices. Options include Apache Kafka, Google Pub/Sub, or Redis, depending on specific requirements and preferences.

NOTE

Keep in mind that the simpler your workflows are, the easier it will be to choose and design a workflow management solution. If you find yourself in a situation where existing solutions do not meet your microservice workflow requirements, it may be beneficial to carefully review and streamline the underlying structure and logic of your workflows before opting for more complex or custom solutions.

Microservices have been making a substantial impact across diverse industries, including the financial sector. A key driver behind this is the emergence of FinTech firms, leveraging cutting-edge technologies and design patterns to create disruptive financial solutions. By segmenting their applications into smaller microservices, FinTech firms gain the required agility to develop, update, and deploy individual functionalities independently.

Meanwhile, traditional financial institutions are embracing microservices as part of their digital transformation and reengineering endeavors aimed at fostering innovation while adapting to the evolving technological landscape. A prominent trend in this realm is the advent of [platform and open banking](#), wherein banks evolve into interconnected ecosystems of financial services. This often involves establishing strategic partnerships with FinTech firms to integrate and deploy their offerings seamlessly within the traditional banking infrastructure.

Through the utilization of microservices, banks can initiate new collaborations with FinTech firms by creating isolated microservices for each distinct application, facilitating streamlined integration and enabling rapid deployment of innovative financial solutions.⁶

Machine Learning Workflows

A machine learning project involves applying scientific techniques and algorithms to analyze and detect patterns from the data, build predictive models, or automate decision-making processes. These projects typically include stages such as data collection, preprocessing, model selection, training, testing, evaluation, and deployment. Given their structured and logical nature, ML projects are often organized as data workflows to ensure their systematic and effective execution as well as optimal data and lifecycle management.⁷

In real-world scenarios, ML workflows can be designed in multiple ways, depending on the unique business needs, data characteristics, and technical requirements. To provide a basic perspective, I will divide the stages involved in an ML workflow into three categories: data related, modeling related, and deployment related. Let's explore each category in some detail.

Data-related steps:

Data extraction

This very first step involves identifying and extracting all required data from various sources, such as databases, APIs, or files.

Quality checks

Once the data is extracted, quality checks are performed to ensure multiple quality dimensions such as accuracy, validity, completeness, timeliness, and many more.

Preprocessing

Once quality checks are completed, preprocessing steps are applied to get the data ready for model training. This includes tasks such as feature engineering, scaling, normalization, encoding, embedding, enrichment, imputation, and many more.

Modeling steps:

Model selection

In this phase, you choose the appropriate machine learning algorithms, models, and optimization techniques based on the nature of your business problem and needs, data quality attributes, and performance requirements.

Training

The selected model is trained on the preprocessed data to learn meaningful patterns from the data and achieve generalizability.

Evaluation

Once trained, the model's performance is evaluated using various metrics and techniques to assess its ability to generalize to new, unseen data.

Model deployment steps:

Deployment

After successful evaluation, the trained model is packaged and deployed into production or operational environments, where it can process requests for making predictions or classifications on new data.

Serving

The deployed model is exposed to its final consumers via APIs or other interfaces to serve predictions in real-time or batch mode, depending on the business requirements at hand.

Model feedback

Continuous monitoring and feedback mechanisms are put in place to assess the model's performance in production, collect feedback from users, and introduce improvements or updates as necessary.

For production ML workflows, additional features are often required. First, a *model registry* is often implemented to store and version persistent various ML workflow steps, including their code, parameters, and data output. This is useful as it enables a business to keep track of historical workflows, ensure point-in-time reproducibility, share ML models across teams, and ensure compliance and transparency.

Second, a highly desired feature of ML workflows is *checkpointing*. This involves periodically saving the workflow's state—including model parameters, data processing stages, and execution context—to persistent storage. In case of a failure, this allows the workflow to reload and resume from the last saved checkpoint.

Third, ML data modeling often requires a specialized, analytical approach. *Feature stores* represent a formal method for implementing this. A feature store is a centralized repository for storing, managing, and serving precomputed and curated machine learning features. Feature stores provide several benefits. First, they enable feature reuse by storing developed features for quick access and sharing across ML models and teams, thereby saving time and fostering efficiency in model development and cross-team cooperation. Second, they ensure feature consistency by centralizing feature definitions and development documentation, which helps maintain uniformity across large organizations. Last, they enhance security and data governance by tracking the data used for model training and deployment.

Fourth, an ML workflow often demands specific computing resources to ensure optimal performance. This can entail leveraging advanced technologies like GPUs for accelerated computation, distributed and parallel computing frameworks for handling large-scale data processing tasks efficiently, and specialized data storage systems such as vector databases, which store data as vector embeddings for fast retrieval and similarity search.

Finally, ensuring the stability, automation, and quality of an ML workflow's deployment and performance requires incorporating software engineering and MLOps best practices. MLOps, short for machine learning operations, encompasses methodologies and tools aimed at automating and optimizing the deployment and management of ML workflows.

Figure 11-14 illustrates the several stages involved in a typical machine learning pipeline as well as the undercurrents that underpin a reliable ML workflow management system.

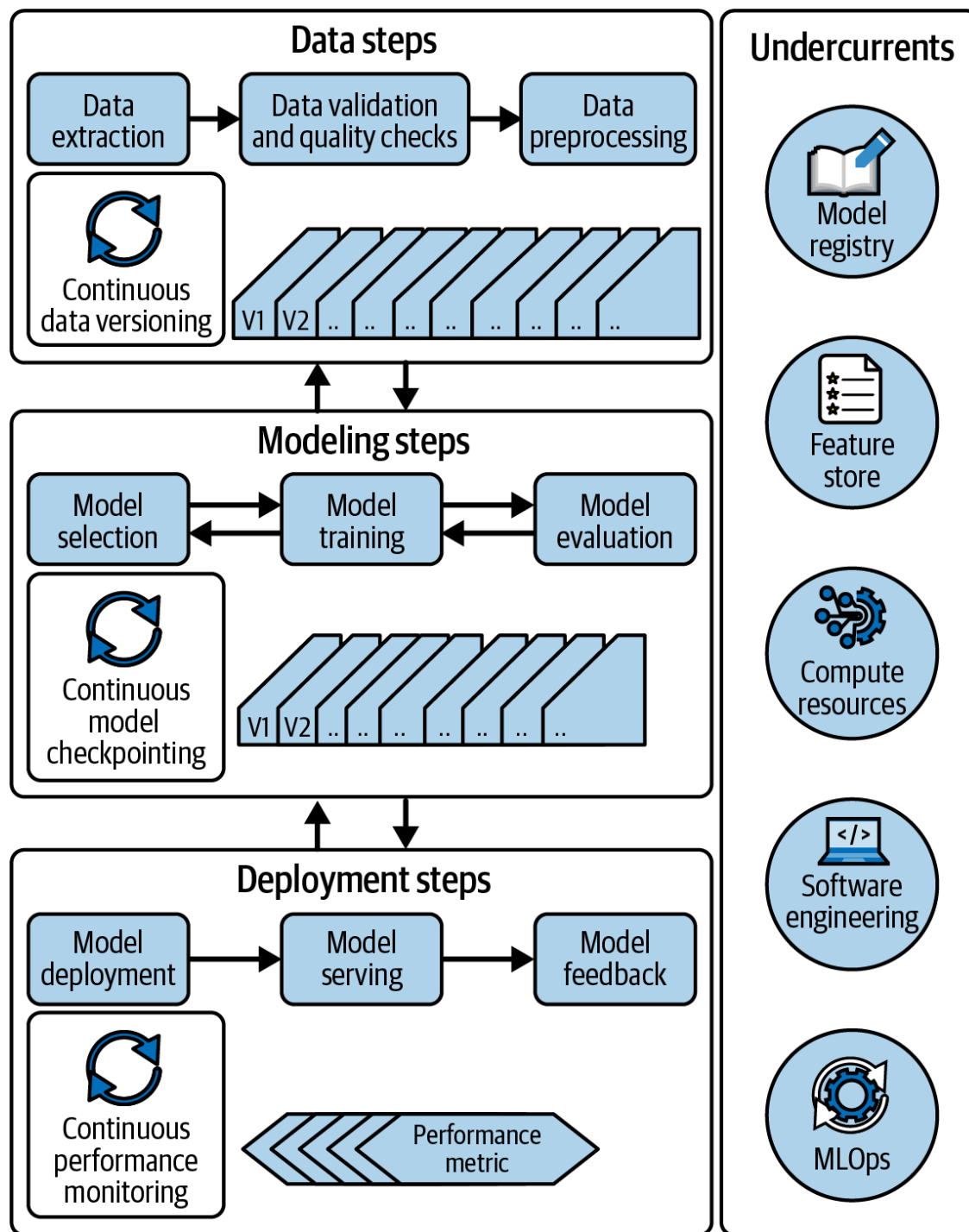


Figure 11-14. ML workflow lifecycle

Designers of ML workflows may need to integrate various domain-specific requirements. For instance, financial institutions must adhere to regulations and guidelines concerning fairness, transparency, and accountability in algorithmic decision-making. An example is the US Equal Credit Opportunity Act of 1974, which mandates creditors to provide applicants with specific reasons for credit denials or altered terms upon request. To comply, an ML workflow must include features to track, retrieve, and explain data, model mechanics, assumptions, and predictions.

In financial markets, the sensitive nature of financial data, coupled with growing public demand for privacy protection and the introduction of stringent privacy regulation worldwide, has imposed restrictions on how and what data can be analyzed, processed, and shared.

To address these issues, financial ML workflows must integrate privacy-preserving features to ensure that sensitive data used in machine learning tasks remains secure and confidential throughout the various phases of the workflow. Key techniques include the following:

Homomorphic encryption

Enables computation on encrypted data without decrypting it, preserving data confidentiality during processing. This technique typically requires complex mathematical operations that can significantly slow down processing speed and increase computational overhead.

Differential privacy

Introduces noise to query results to protect individual data privacy while maintaining statistical accuracy.

Secure multiparty computation

Enables computations across multiple parties without revealing each party's private data to the others. This protocol ensures that each party can contribute their data securely to the computation process while maintaining confidentiality.

Federated learning

Trains machine learning models on decentralized data sources without exchanging raw data, thereby preserving privacy.

Synthetic data generation

Creates artificial data that retains statistical properties of the original dataset while protecting sensitive information.

Implementing these techniques ensures that financial machine learning workflows uphold privacy standards, fostering responsible data usage and enhancing security measures in the finance sector.

The challenges involved in building reliable and high-performance ML models largely revolve around effective data management and ensuring data quality. As a financial data engineer, your responsibility is crucial in establishing robust ML workflows. These workflows are indispensable in today's financial markets, where the integration of ML and AI technologies stands as the primary driver of transformation and innovation.⁸

Summary

This chapter provided a comprehensive examination of financial data workflows and their fundamental concepts. It started by emphasizing the value and need for workflows through the concept of workflow-oriented software architectures. The chapter then defined both data workflows and workflow management systems, highlighting their key features. Finally, it provided an in-depth examination of the main types of data workflows used within the financial sector, offering comprehensive insights into their fundamental concepts and applications. These are ETL, microservices, stream processing, and machine learning workflows.

Congratulations on reaching this point of the book. Your thorough understanding of the foundational concepts of financial data engineering has prepared you for the practical hands-on experience that awaits in the next and final chapter, where you will work on four engaging hands-on projects.

¹ For a comprehensive treatment of data warehouse-oriented ETL, I highly recommend the seminal work of Joe Caserta and Ralph Kimball, *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data* (Wiley, 2013).

- ² For more on ETL patterns, I recommend Vasileios Theodorou, Alberto Abelló, Maik Thiele, and Wolfgang Lehner’s [“Frequent Patterns in ETL Workflows: An Empirical Approach”](#), *Data & Knowledge Engineering* 112 (November 2017): 1–16.
- ³ To learn more about the Lambda and Kappa architectures, I highly recommend James Warren and Nathan Marz’s *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems* (Manning, 2015).
- ⁴ For a good discussion of this topic, I recommend [“Fraud Detection with Apache Kafka, KSQL and Apache Flink”](#), by Kai Waehner. To better understand how to assess the requirements of stream processing systems, I recommend the paper by Michael Stonebraker, Uğur Çetintemel, and Stan Zdonik, [“The 8 Requirements of Real-Time Stream Processing”](#), *ACM SIGMOD Record* 34, no. 4 (December 2005): 42–47.
- ⁵ To learn more about how to use DDD in designing microservices, consult *Microservices Patterns: With Examples in Java* by Chris Richardson (Manning, 2018).
- ⁶ For an interesting case study from the financial sector, see how Danske Bank, the largest bank in Denmark, moved from a monolith codebase into a microservice-oriented architecture, as presented in Antonio Bucchiarone, Nicola Dragoni, Schahram Dustdar, Stephan T. Larsen, and Manuel Mazzara’s [“From Monolithic to Microservices: An Experience Report from the Banking Domain”](#), *IEEE Software* 35, no. 3 (May–June 2018): 50–55.
- ⁷ For a more detailed discussion of the value and the need for ML workflows, I suggest Hui Miao, Ang Li, Larry S. Davis, and Amol Deshpande’s [“Towards Unified Data and Lifecycle Management for Deep Learning”](#), in the *2017 IEEE 33rd International Conference on Data Engineering (ICDE)* (IEEE, 2017): 571–582.
- ⁸ To learn more about the data-related challenges encountered in ML workflow design, I recommend Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich’s [“Data Lifecycle Challenges in Production Machine Learning: A Survey”](#), *ACM SIGMOD Record* 47, no. 2 (December 2018): 17–28.